

Week 3 - Day 13b: Introduction to Graphs

A graph is a data structure which consists of vertices and edges. Vertices may also be known as nodes. More formally, a simple graph is a pair of sets (V, E) , where V is an arbitrary non-empty finite set, whose elements are called vertices, and E is a set of pairs of elements of V , which we call edges.

Mathematically, we can express this as:

$$G=(V,E)$$

where $V=\{v_1, v_2, \dots, v_n\}$ is a set of vertices and $E \subseteq \{\{u, v\}: u, v \in V \text{ and } u \neq v\}$ is the set of edges.

We will use the `networkx` and `matplotlib` library to draw graphs in Python. Let's install and import them.

```
# Install the libraries
%pip install networkx matplotlib

import networkx as nx

def draw(graph):
    nx.draw(nx.DiGraph(graph), with_labels=True, node_size=1000,
            node_color='#cfc8f4', arrowsize=40)
```

Exercise 1: Drawing and Adjacency Dictionary []

1.1 Draw the directed graph (on paper) that has the following edge list: `[[1, 2], [1, 0], [2, 0]]`.

1.2 What is the adjacency dictionary of this graph?

```
# TODO: Your code here
```

1.3 Draw the directed graph (on paper) that has the following adjacency dict: `{0: [1, 2, 3], 1: [2], 2: [], 3: []}`

1.4 What is the list of edges of this graph?

```
# TODO: Your code here
```

Exercise 2: Graph Representations []

2.1 Write a function `to_adj_dict` that takes a list of edges `edge_list` and returns the corresponding adjacency dictionary. Assume that the graph is *directed*.

```
# TODO: Your code here
def to_adj_dict(edge_list):
    pass

# Use this cell to test your code
assert to_adj_dict([]) == {}
assert to_adj_dict([(0, 1), (0, 2), (1, 2)]) == {0: [1, 2], 1: [2], 2: []}
assert to_adj_dict([(1, 2), (2, 3), (3, 1)]) == {1: [2], 2: [3], 3: [1]}
assert to_adj_dict([(0, 1), (1, 2), (2, 3)]) == {0: [1], 1: [2], 2: [3], 3: []}

print("All test passed. ☐")
```

2.2 Write a function `to_edge_list` that takes an adjacency dictionary `adj_dict` and returns the corresponding list of edges.

```
# TODO: Your code here
def to_edge_list(adj_dict):
    pass

# Use this cell to test your code
assert to_edge_list({}) == []
assert to_edge_list({0: [1, 2], 1: [2], 2: []}) == [(0, 1), (0, 2), (1, 2)]
assert to_edge_list({1: [2], 2: [3], 3: [1]}) == [(1, 2), (2, 3), (3, 1)]
assert to_edge_list({0: [1], 1: [2], 2: [3], 3: []}) == [(0, 1), (1, 2), (2, 3)]
assert to_edge_list({0: [1]}) == [(0, 1)]

print("All tests passed. ☐")
```

Cool! Now you can convert between adjacency dictionaries and edge lists as often as you want.

```
graph = {0: [1, 2], 1: [2], 2: []}
print(to_adj_dict(to_edge_list(graph))) # Will print the original graph
```

Exercise 3: Cycle Graph [] []

A cycle graph is a graph that consists of a single cycle, i.e. a sequence of vertices connected by edges, where the last vertex is connected to the first vertex.

cycle-graph.png

2.1 Write a function that takes a parameter `n` and returns the adjacency dictionary of the cycle graph on `n` nodes.

```
# TODO: Your code here
def cycle_graph(n):
    pass

# Test cases for cycle_graph function
assert cycle_graph(1) == {0: [0]}
assert cycle_graph(2) == {0: [1], 1: [0]}
assert cycle_graph(3) == {0: [1], 1: [2], 2: [0]}
assert cycle_graph(4) == {0: [1], 1: [2], 2: [3], 3: [0]}
assert cycle_graph(5) == {0: [1], 1: [2], 2: [3], 3: [4], 4: [0]}

print("All tests passed. ☑")
```

2.2 You can also check your solution by using the `draw` function (defined at the beginning of the sheet). Try calling it by passing the adjacency dictionary of some cycle graph (generated by `cycle_graph`).

```
# TODO: Your code here
```

Exercise 4: Complete Graph [□□□]

A **complete graph** is a graph where all nodes are connected.

complete-graph.png

4.1 Write a function that takes a parameter `n` and returns the complete graph on `n` nodes as an **edge list**.

```
# TODO: Your code here
def complete_graph(n):
    pass

# Test cases for complete_graph function
assert complete_graph(1) == []
assert complete_graph(2) == [(0, 1), (1, 0)]
assert complete_graph(3) == [(0, 1), (0, 2), (1, 0), (1, 2), (2, 0), (2, 1)]

print("All tests passed. ☑")
```

4.1 Finally, draw an example of a complete graph with 5 nodes using the `draw` function.

```
# TODO: Your code here
```

Exercise 5: Check Neighbors in a Graph [□□□]

You are given a graph and two vertices. Check if they are neighbors. Recall that two vertices are neighbors if they are connected by an edge.

5.1 Implement the `neighbours` function to check if vertex `y` is a neighbor of vertex `x` in graph `G`, i.e. if there is an edge `(x, y)` in `G`. Assume that the graph is directed and that `G` is represented as an adjacency dictionary.

```
# TODO: Your code here
def neighbours(G, x, y):
    pass

# Use this cell to test your code
G = {0: [1, 2, 3], 1: [0], 2: [0, 4, 5], 3: [0], 4: [2, 5], 5: [2, 4]}
assert neighbours(G, 2, 5) == True
assert neighbours(G, 5, 2) == True
assert neighbours(G, 3, 2) == False
assert neighbours(G, 0, 4) == False

G = {0: [1, 3], 1: [], 2: [0, 4, 5], 3: [0], 4: [2, 5], 5: [2, 4]}
assert neighbours(G, 0, 2) == False

G = {0: [1, 2, 3], 1: [0], 2: [0, 4, 5], 3: [0], 4: [2, 5], 5: [2, 4]}
assert neighbours(G, 2, 2) == False

print("All tests passed. ☐")
```

Exercise 6: Check Distance-Two Neighbors in a Graph

☐☐☐☐

You are given a graph and two vertices. Check if they are distance two apart. For example, in the picture below, vertices 1 and 2 are distance two apart, while 3 and 4 are not, and 4 and 5 are not because they are neighbors.

Graph

For instance, this problem can help determine whether people `x` and `y` have mutual friends.

For example, for `G = {0:[1,2,3], 1:[0], 2:[0,4,5], 3:[0], 4:[2,5], 5:[2,4]}`, `x = 2`, `y = 3`, it should return `True`.

Be sure you have checked all the corner cases.

6.1 Implement the `almost_neighbours` function to check if `y` is distance two apart from `x` in graph `G`. Assume that the graph is directed and that `G` is represented as an adjacency dictionary.

```
# TODO: Your code here
def almost_neighbours(G, x, y):
    pass

# Test cases
G = {0: [1, 2, 3], 1: [0], 2: [0, 4, 5], 3: [0], 4: [2, 5], 5: [2, 4]}
assert almost_neighbours(G, 2, 3) == True
```

```
G = {0: [1, 2, 3], 1: [0], 2: [0, 4, 5], 3: [0], 4: [2, 5], 5: [2, 4]}
assert almost_neighbours(G, 3, 2) == True

G = {0: [1, 2, 3], 1: [0], 2: [0, 4, 5], 3: [0], 4: [2, 5], 5: [2, 4]}
assert almost_neighbours(G, 1, 5) == False

G = {0: [1, 2, 3], 1: [0], 2: [0, 4, 5], 3: [0], 4: [2, 5], 5: [2, 4]}
assert almost_neighbours(G, 3, 4) == False

G = {0: [1, 3], 1: [], 2: [0, 4, 5], 3: [0], 4: [2, 5], 5: [2, 4]}
assert almost_neighbours(G, 0, 2) == False

G = {0: [1, 2, 3], 1: [0], 2: [0, 4, 5], 3: [0], 4: [2, 5], 5: [2, 4]}
assert almost_neighbours(G, 0, 0) == False

print("All tests passed. ☐")
```

Congratulations ☐